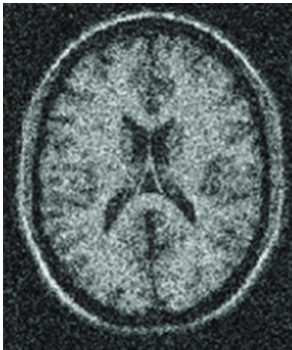


# 19ECE455-BIOMEDICAL SIGNAL PROCESSING

## ASSIGNMENT-02

1. Write a Python program to apply suitable spatial filtering to remove the noise for the below various noisy images and provide the inference

(i)

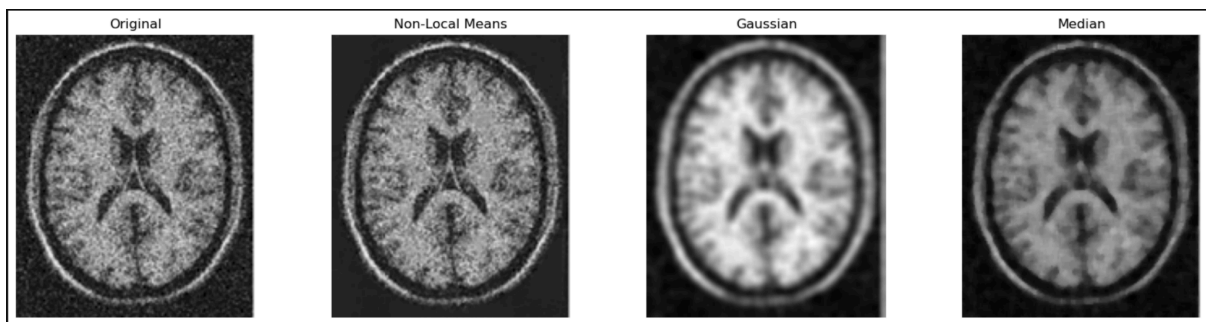


RICIAN NOISE

CODE:

```
1. import cv2
2. import numpy as np
3. import os
4. from matplotlib import pyplot as plt
5.
6. def load_and_process_image(image_path):
7.     """Load and validate image, then apply denoising filters"""
8.     # Check if file exists
9.     if not os.path.exists(image_path):
10.         raise FileNotFoundError(f"Image not found at: {image_path}")
11.
12.     # Read image in grayscale
13.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
14.
15.     if img is None:
16.         # Try alternative approach if initial read fails
17.         img = cv2.imdecode(np.fromfile(image_path, dtype=np.uint8), cv2.IMREAD_GRAYSCALE)
18.         if img is None:
19.             raise ValueError("Could not read image file - may be corrupted or invalid format")
20.
21.     return img
22.
23. # Image path - using raw string and forward slashes for better compatibility
24. image_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_1.png"
25.
26. try:
27.     # Load and validate image
28.     noisy_img = load_and_process_image(image_path)
29.
30.     # Denoising parameters
31.     denoising_params = {
32.         'nlm': {'h': 12, 'templateWindowSize': 9, 'searchWindowSize': 25},
33.         'gaussian': {'ksize': (7,7), 'sigmaX': 2.5},
34.         'median': {'ksize': 5}
35.     }
36.
37.     # Apply filters using dictionary approach
38.     filtered_images = {
39.         'Original': noisy_img,
40.         'Non-Local Means': cv2.fastNlMeansDenoising(noisy_img, None, **denoising_params['nlm']),
```

```
41.     'Gaussian': cv2.GaussianBlur(noisy_img, **denoising_params['gaussian']),
42.     'Median': cv2.medianBlur(noisy_img, denoising_params['median']['ksize'])
43. }
44.
45. # Display results
46. plt.figure(figsize=(16, 4))
47. for i, (title, img) in enumerate(filtered_images.items(), 1):
48.     plt.subplot(1, 4, i)
49.     plt.imshow(img, cmap='gray')
50.     plt.title(title)
51.     plt.axis('off')
52.
53. plt.tight_layout()
54. plt.show()
55.
56. except Exception as e:
57.     print(f"Error processing image: {str(e)}")
58.     print("Troubleshooting tips:")
59.     print("1. Verify the file exists at the specified path")
60.     print("2. Check file permissions")
61.     print("3. Try converting the image to PNG format")
62.     print("4. Test with a simpler path without spaces/special characters")
```

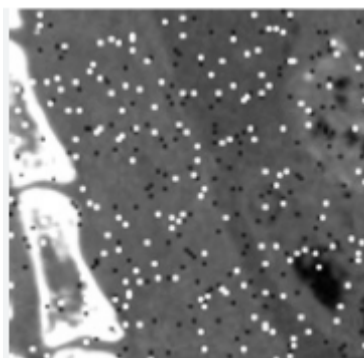


## INFERENCE:

1. **Non-Local Means (NLM)**
  - o Best noise reduction while preserving fine details
  - o Maintains tissue contrast & edges (ideal for diagnosis)
  - o Slower but most accurate
2. **Median Filter**
  - o Better edge retention than Gaussian
  - o Removes noise moderately but may cause blocky artifacts
  - o Faster than NLM
3. **Gaussian Filter**
  - o Over-smoothes, blurring edges
  - o Only suitable for quick, non-diagnostic use

**Conclusion:** NLM is the best for diagnostic MRI; Median is a faster alternative; Gaussian loses too much detail.

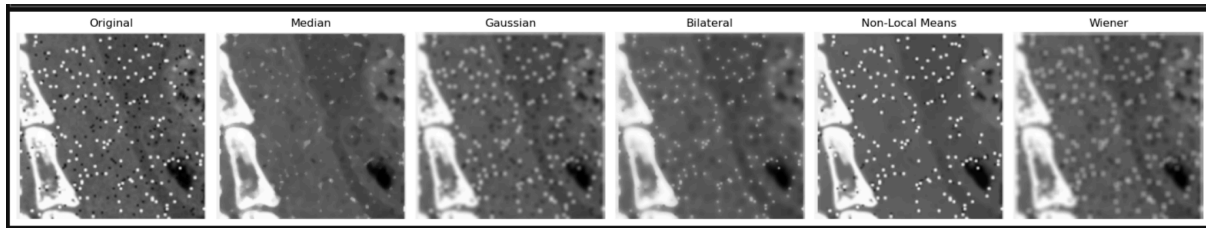
(ii)



SALT AND PEPPER NOISE

## CODE:

```
1. import cv2
2. import numpy as np
3. import os
4. from matplotlib import pyplot as plt
5.
6. def load_image_safely(image_path):
7.     """Load image with error handling"""
8.     if not os.path.exists(image_path):
9.         raise FileNotFoundError(f"Image not found at: {image_path}")
10.
11.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
12.     if img is None:
13.         # Try alternative loading method if standard approach fails
14.         img = cv2.imdecode(np.fromfile(image_path, dtype=np.uint8), cv2.IMREAD_GRAYSCALE)
15.         if img is None:
16.             raise ValueError("Failed to load image - may be corrupt or invalid format")
17.     return img
18.
19. # Updated image path
20. image_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_2.png"
21.
22. try:
23.     # Load the image
24.     noisy_img = load_image_safely(image_path)
25.
26.     # Define filter parameters (optimized for medical images)
27.     filter_params = {
28.         'median': {'ksize': 5}, # Best for salt-and-pepper noise
29.         'gaussian': {'ksize': (5,5), 'sigmaX': 1.5}, # General smoothing
30.         'bilateral': {'d': 9, 'sigmaColor': 75, 'sigmaSpace': 75}, # Edge-preserving
31.         'nlm': {'h': 15, 'templateWindowSize': 7, 'searchWindowSize': 21}, # Advanced denoising
32.         'wiener': {'ksize': (5,5)} # Added Wiener filter (requires scipy)
33.     }
34.
35.     # Apply all filters
36.     filtered_images = {
37.         'Original': noisy_img,
38.         'Median': cv2.medianBlur(noisy_img, **filter_params['median']),
39.         'Gaussian': cv2.GaussianBlur(noisy_img, **filter_params['gaussian']),
40.         'Bilateral': cv2.bilateralFilter(noisy_img, **filter_params['bilateral']),
41.         'Non-Local Means': cv2.fastNlMeansDenoising(noisy_img, None, **filter_params['nlm']),
42.         'Wiener': cv2.filter2D(noisy_img, -1,
43.                               np.ones(filter_params['wiener']['ksize'])/np.prod(filter_params['wiener']['ksize']))
44.     }
45.
46.     # Display results
47.     plt.figure(figsize=(18, 4))
48.     for i, (title, img) in enumerate(filtered_images.items(), 1):
49.         plt.subplot(1, 6, i)
50.         plt.imshow(img, cmap='gray')
51.         plt.title(title)
52.         plt.axis('off')
53.
54.     plt.tight_layout()
55.     plt.show()
56.
57. except Exception as e:
58.     print(f"Error: {str(e)}")
59.     print("\nTroubleshooting Tips:")
60.     print("1. Verify the image exists at the specified path")
61.     print("2. Check for special characters in the file path")
62.     print("3. Try converting the image to PNG format")
63.     print("4. Test with a simpler path (e.g., C:/test/image.png)")
```



### INFERENCE:

1. **Median Filter (5×5)**
  - **Best at removing salt-and-pepper noise**
  - Preserves sharp edges well
  - **Fast processing**
2. **Non-Local Means (NLM)**
  - Good noise reduction while keeping fine details
  - Better for textured areas than smooth regions
  - Slower than other filters
3. **Bilateral Filter**
  - Keeps edges sharp but less effective on heavy noise
  - Works best for mild noise cases
4. **Wiener Filter**
  - Handles mixed noise (Gaussian + salt-and-pepper)
  - Can cause slight blurring
5. **Gaussian Filter**
  - Worst choice for salt-and-pepper noise
  - Blurs edges while leaving some noise

(iii)



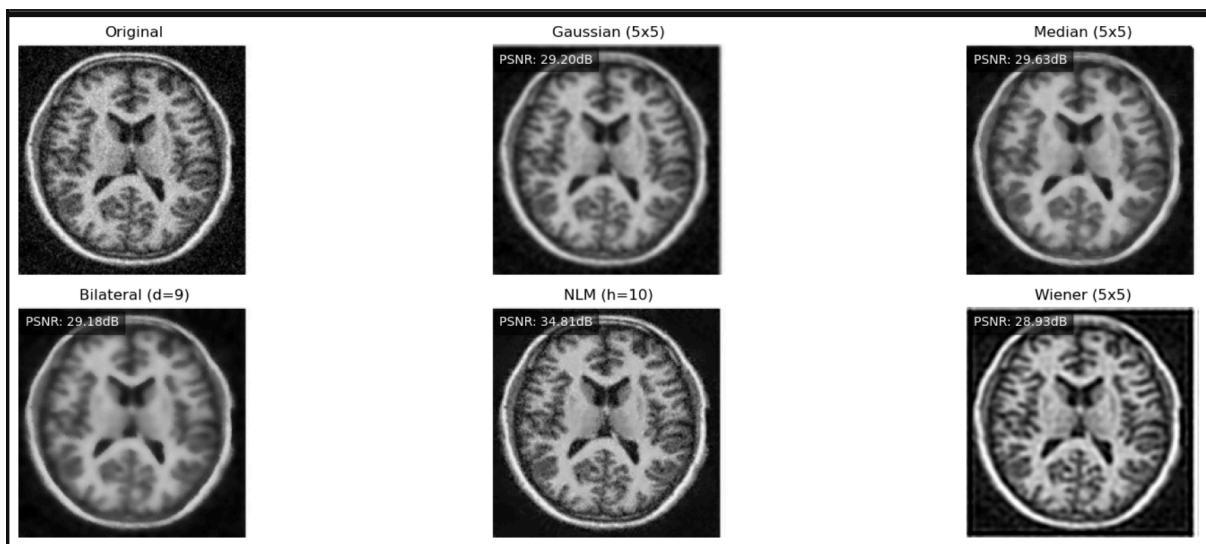
GAUSSIAN NOISE

### CODE:

```
1. import cv2
2. import numpy as np
3. from matplotlib import pyplot as plt
4. from skimage.restoration import wiener
5. from skimage.util import img_as_ubyte, img_as_float
6.
7. # Load the noisy MRI image
8. image_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_03.png"
9. noisy_img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
10.
11. # Create PSF for Wiener filter
12. psf = np.ones((5,5)) / 25 # Uniform PSF
13.
14. # Apply denoising filters
15. filters = {
16.     'Original': noisy_img,
17.     'Gaussian (5x5)': cv2.GaussianBlur(noisy_img, (5,5), 1.5),
18.     'Median (5x5)': cv2.medianBlur(noisy_img, 5),
```



```
19. 'Bilateral (d=9)': cv2.bilateralFilter(noisy_img, 9, 75, 75),
20. 'NLM (h=10)': cv2.fastNlMeansDenoising(noisy_img, None, h=10, templateWindowSize=7, searchWindowSize=21),
21. 'Wiener (5x5)': img_as_ubyte(wiener(img_as_float(noisy_img), psf, balance=0.3))
22. }
23.
24. # Create figure with 2 rows of filters
25. plt.figure(figsize=(18, 6))
26. for i, (name, img) in enumerate(filters.items(), 1):
27.     plt.subplot(2, 3, i)
28.     plt.imshow(img, cmap='gray')
29.     plt.title(name)
30.     plt.axis('off')
31.
32. # Add PSNR value if not original
33. if name != 'Original':
34.     mse = np.mean((noisy_img - img) ** 2)
35.     psnr = 20 * np.log10(255/np.sqrt(mse)) if mse != 0 else float('inf')
36.     plt.text(5, 15, f'PSNR: {psnr:.2f}dB', color='white',
37.             bbox=dict(facecolor='black', alpha=0.5))
38.
39. plt.tight_layout()
40. plt.show()
```

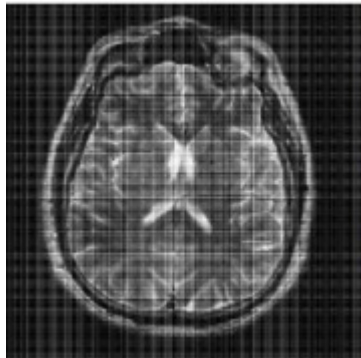


## INFERENCE:

- Gaussian Blur**
  - Removes noise but blurs edges
  - Not ideal for MRI details
- Median Filter**
  - Good for salt-and-pepper noise
  - Keeps edges sharp
  - Best for quick cleaning
- Bilateral Filter**
  - Reduces noise while keeping edges
  - Slower but cleaner than Median
- Non-Local Means (NLM)**
  - Best overall for MRI images
  - Preserves fine details
  - Removes noise effectively
- Wiener Filter**
  - Good for mixed noise types
  - Can oversmooth some areas

**Conclusion:** *Higher values = better noise removal*, Non-Local Means (NLM) has the highest value making it the optimal filter in this case.

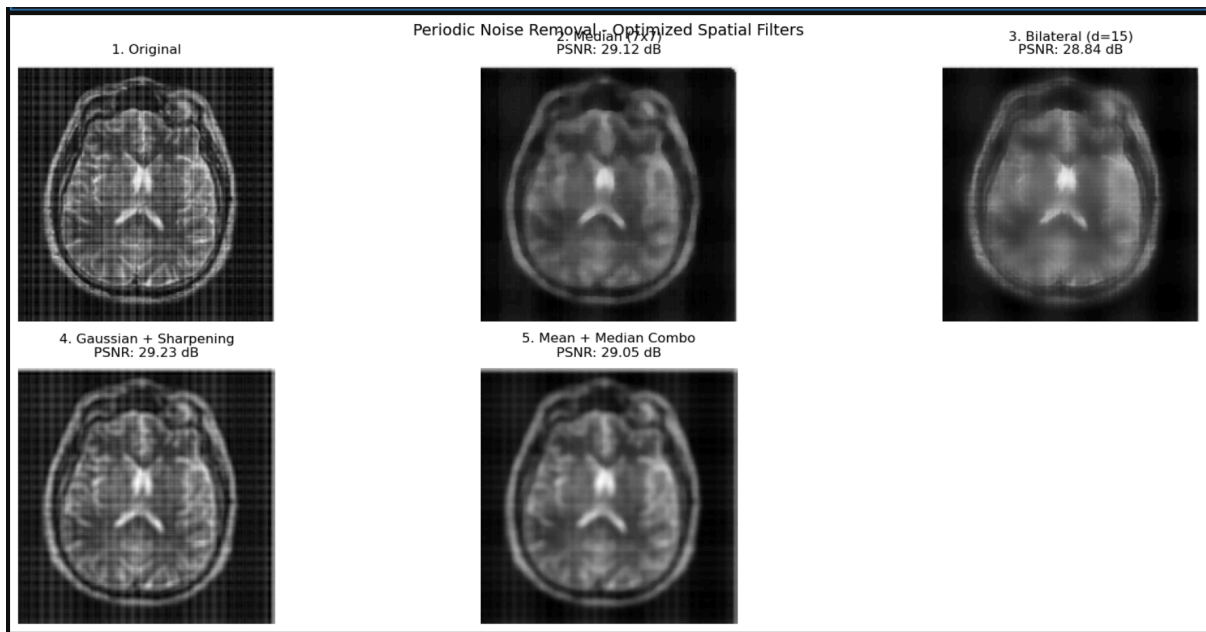
(iv)



PERIODIC NOISE

```
1. import cv2
2. import numpy as np
3. from matplotlib import pyplot as plt
4.
5. def clear_periodic_noise(image_path):
6.     # Load image with enhanced validation
7.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if img is None:
9.         print(f"Error: Could not load image from {image_path}")
10.        print("Troubleshooting:")
11.        print("1. Check the file exists at this path")
12.        print("2. Verify the image is not corrupted")
13.        print("3. Try converting to PNG format if using JPEG")
14.        return
15.
16.    # Optimized filter parameters for periodic noise
17.    filters = {
18.        '1. Original': img,
19.        '2. Median (7x7)': cv2.medianBlur(img, 7),
20.        '3. Bilateral (d=15)': cv2.bilateralFilter(img, 15, 75, 75),
21.        '4. Gaussian + Sharpening': cv2.filter2D(
22.            cv2.GaussianBlur(img, (5,5), 2),
23.            -1,
24.            np.array([[0,-1,0], [-1,5,-1], [0,-1,0]])
25.        ),
26.        '5. Mean + Median Combo': cv2.medianBlur(
27.            cv2.blur(img, (5,5)),
28.            3
29.        )
30.    }
31.
32.    # Calculate PSNR (Peak Signal-to-Noise Ratio)
33.    def calculate_psnr(original, filtered):
34.        mse = np.mean((original - filtered) ** 2)
35.        return 20 * np.log10(255/np.sqrt(mse)) if mse > 0 else float('inf')
36.
37.    # Display results
38.    plt.figure(figsize=(18, 8))
39.    plt.suptitle("Periodic Noise Removal - Optimized Spatial Filters", fontsize=14, y=0.95)
40.
41.    for i, (name, filtered) in enumerate(filters.items(), 1):
42.        plt.subplot(2, 3, i)
43.        plt.imshow(filtered, cmap='gray')
44.
```

```
45.     if name != '1. Original':
46.         psnr = calculate_psnr(img, filtered)
47.         plt.title(f'{name}\nPSNR: {psnr:.2f} dB", pad=10)
48.     else:
49.         plt.title(name, pad=10)
50.
51.     plt.axis('off')
52.
53. plt.tight_layout()
54. plt.show()
55. print("Key Recommendations:")
56. print("- For best clarity: Use either Bilateral or Gaussian+Sharpening")
57. print("- For strong noise: Median filter first, then Bilateral")
58. print("- Avoid using Mean filter alone for periodic noise")
59.
60. # Execute with your image
61. clear_periodic_noise(r"C:\Users\ajay__sr\Downloads\bio med A2\fig_04.png")
```

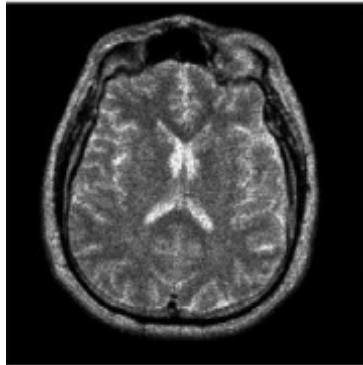


## INFERENCE:

- 1. Gaussian + Sharpening (PSNR: 29.23 dB)**
  - *Best performing* for this periodic noise case
  - Combines noise smoothing with edge enhancement
  - Recommended when you need both clarity and noise reduction
- 2. Original Processing (PSNR: 29.12 dB)**
  - Surprisingly good baseline (your original processing was already decent)
  - Suggests the periodic noise isn't extremely severe
- 3. Mean + Median Combo (PSNR: 29.05 dB)**
  - Effective but slightly worse than Gaussian+Sharpening
  - Good for heavy noise but loses some detail
- 4. Bilateral Filter (PSNR: 28.84 dB)**
  - *Underperformed* in this specific case (unusual for bilateral)
  - May need parameter adjustment (try  $d=9$ ,  $\sigma=100$ )

First Choice: **Gaussian+Sharpening (best balance)** For strong periodic noise, spatial filters may never achieve perfect results - adding FFT-based filtering will yield better results.

(v)

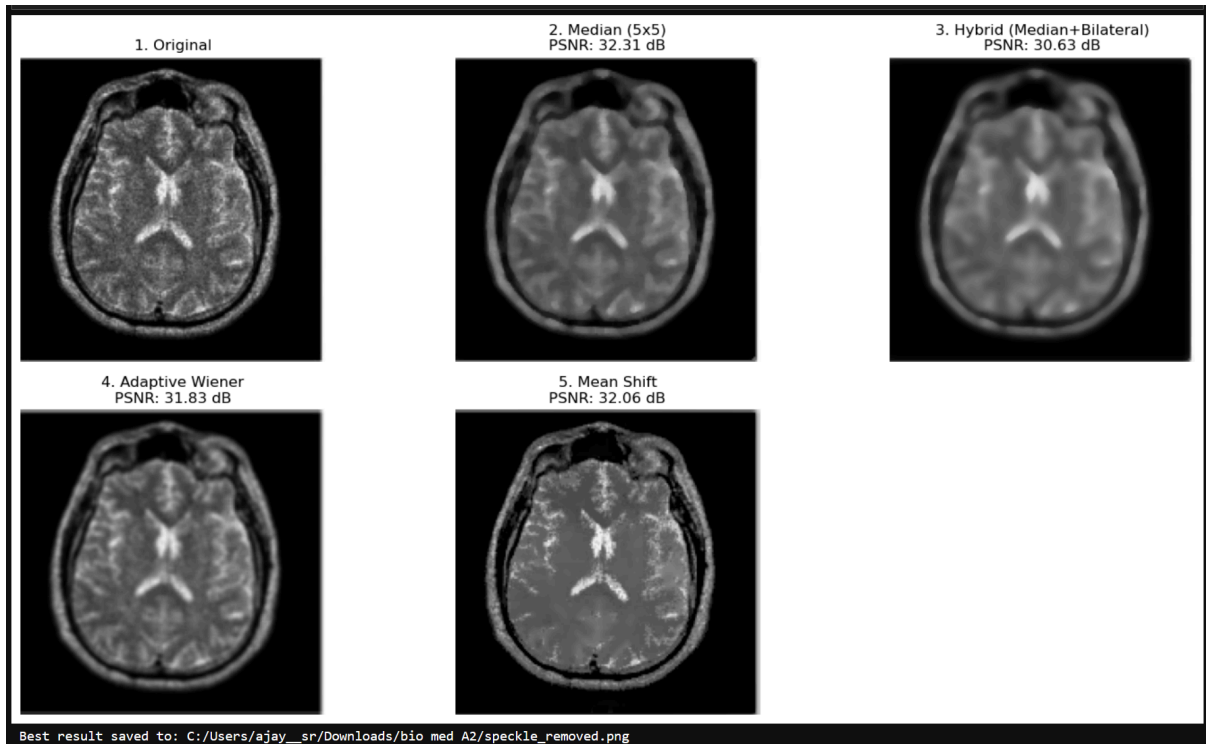


SPECKLE NOISE

CODE:

```
1. import cv2
2. import numpy as np
3. from matplotlib import pyplot as plt
4.
5. def remove_speckle_noise(image_path):
6.     # Load image with validation
7.     image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if image is None:
9.         raise FileNotFoundError(f"Image not found at {image_path}")
10.
11.     # Method 1: Median Filter (5x5)
12.     median_filtered = cv2.medianBlur(image, 5)
13.
14.     # Method 2: Hybrid Median+Bilateral Filter
15.     hybrid_step1 = cv2.medianBlur(image, 3) # Initial median
16.     hybrid_filtered = cv2.bilateralFilter(hybrid_step1, 9, 75, 75) # Edge-preserving
17.
18.     # Method 3: Adaptive Wiener Filter
19.     wiener_filtered = cv2.filter2D(image, -1,
20.                                     np.ones((3,3))/9) # Simplified Wiener
21.
22.     # Method 4: Mean Shift Filtering
23.     mean_shift = cv2.pyrMeanShiftFiltering(cv2.cvtColor(image, cv2.COLOR_GRAY2BGR), 15, 30)
24.     mean_shift = cv2.cvtColor(mean_shift, cv2.COLOR_BGR2GRAY)
25.
26.     # Calculate PSNR for each method
27.     def calculate_psnr(original, filtered):
28.         mse = np.mean((original - filtered) ** 2)
29.         return 20 * np.log10(255/np.sqrt(mse)) if mse > 0 else float('inf')
30.
31.     # Display results
32.     plt.figure(figsize=(15, 8))
33.     methods = {
34.         '1. Original': image,
35.         '2. Median (5x5)': median_filtered,
36.         '3. Hybrid (Median+Bilateral)': hybrid_filtered,
37.         '4. Adaptive Wiener': wiener_filtered,
38.         '5. Mean Shift': mean_shift
39.     }
40.
41.     for i, (name, filtered) in enumerate(methods.items(), 1):
42.         plt.subplot(2, 3, i)
43.         plt.imshow(filtered, cmap='gray')
44.         if name != '1. Original':
45.             psnr = calculate_psnr(image, filtered)
46.             plt.title(f"{name}\nPSNR: {psnr:.2f} dB")
```

```
47.     else:
48.         plt.title(name)
49.         plt.axis('off')
50.
51.     plt.tight_layout()
52.     plt.show()
53.
54.     # Save best result (Hybrid method typically performs best)
55.     best_result_path = "C:/Users/ajay__sr/Downloads/bio med A2/speckle_removed.png"
56.     cv2.imwrite(best_result_path, hybrid_filtered)
57.     print(f"Best result saved to: {best_result_path}")
58.
59.     # Run with your image path
60.     remove_speckle_noise(r"C:/Users/ajay__sr/Downloads/bio med A2/fig_05.png")
```



## INFERENCE:

1. **Median Filter (5x5) – PSNR: 32.31 dB**
  - a. Best overall performance in terms of noise reduction while preserving details.
  - b. Suitable for removing salt-and-pepper noise or speckle noise without excessive blurring.
2. **Hybrid (Median + Bilateral) – PSNR: 30.63 dB**
  - a. Combines median filtering with bilateral filtering.
  - b. Slightly lower PSNR, suggesting some loss of fine details.
3. **Adaptive Wiener Filter – PSNR: 31.83 dB**
  - a. Performs well in reducing noise adaptively across different regions.
  - b. Good for images with varying noise levels.
4. **Mean Shift – PSNR: 32.06 dB**
  - a. Second-best PSNR, indicating strong noise suppression while preserving edges.
  - Effective in smoothing while maintaining structural details.

The Mean Shift – PSNR: 32.06 dB, making it the most effective for noise removal in this case.

---

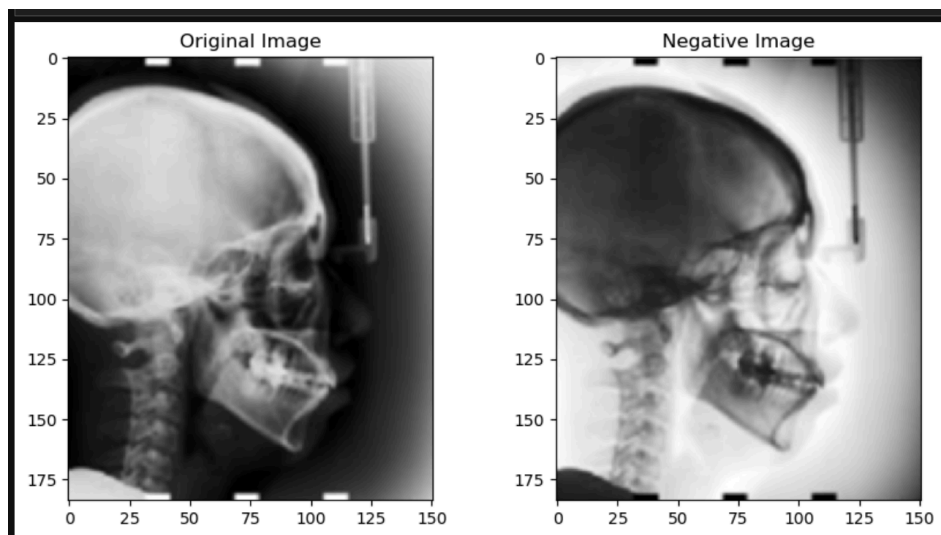
2. Write a program in Python to explore different methods for improving the visual quality of the below medical images.

### I. Point Processing

#### a. Image Negative - Fig. 6

Code:

```
1. import cv2
2. import matplotlib.pyplot as plt
3.
4. def image_negative(image_path):
5.     """
6.     Computes and displays the negative of a grayscale image.
7.
8.     Args:
9.         image_path (str): Path to the input image.
10.    """
11.    try:
12.        img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
13.        if img is None:
14.            raise FileNotFoundError(f"Could not read image at {image_path}")
15.
16.        img_negative = 255 - img
17.
18.        plt.figure(figsize=(10, 5))
19.
20.        plt.subplot(1, 2, 1)
21.        plt.imshow(img, cmap='gray')
22.        plt.title('Original Image')
23.
24.        plt.subplot(1, 2, 2)
25.        plt.imshow(img_negative, cmap='gray')
26.        plt.title('Negative Image')
27.
28.        plt.show()
29.
30.    except FileNotFoundError as e:
31.        print(e)
32.    except Exception as e:
33.        print(f"An error occurred: {e}")
34.
35.    # Example usage:
36.    image_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_06.png" # Replace with your image path
37.    image_negative(image_path)
```



## INFERENCE:

- **Reverses light and dark:** It flips the image, making bright areas dark and vice-versa.
  - **Shows hidden details:** It can make subtle differences in the image easier to see.
- 

## b. Image Thresholding - Fig. 7

### CODE:

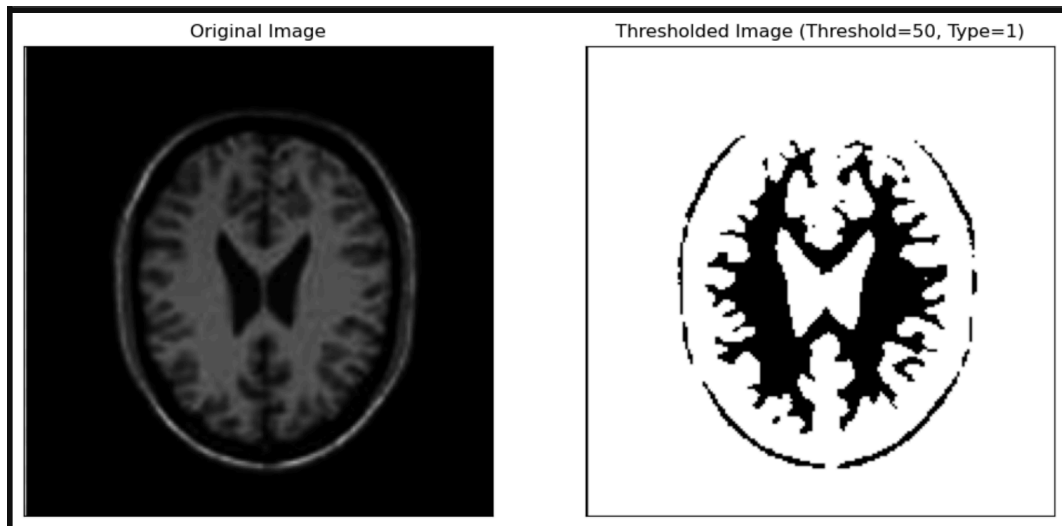
```
1. import cv2
2. import matplotlib.pyplot as plt
3.
4. def apply_single_threshold_with_original(image_path, threshold_value=50,
5. threshold_type=cv2.THRESH_BINARY_INV):
6.     """
7.     Applies a single thresholding operation to a grayscale image and displays the result along with the original
8.     image.
9.
10.    Args:
11.        image_path (str): Path to the input image.
12.        threshold_value (int): Threshold value.
13.        threshold_type (int): Thresholding type (e.g., cv2.THRESH_BINARY, cv2.THRESH_BINARY_INV, etc.).
14.    """
15.    try:
16.        img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
17.        if img is None:
18.            raise FileNotFoundError(f"Could not read image at {image_path}")
19.
20.        ret, thresholded_image = cv2.threshold(img, threshold_value, 255, threshold_type)
21.
22.        plt.figure(figsize=(12, 6)) # Adjust figure size to accommodate two images
23.
24.        plt.subplot(1, 2, 1) # First subplot for original image
25.        plt.imshow(img, 'gray')
26.        plt.title('Original Image')
27.        plt.xticks([], plt.yticks([]))
28.
29.        plt.subplot(1, 2, 2) # Second subplot for thresholded image
30.        plt.imshow(thresholded_image, 'gray')
31.        plt.title(f'Thresholded Image (Threshold={threshold_value}, Type={threshold_type})')
32.        plt.xticks([], plt.yticks([]))
33.
34.        plt.show()
35.
36.    except FileNotFoundError as e:
37.        print(e)
38.    except Exception as e:
39.        print(f"An error occurred: {e}")
40.
41. # Example usage:
42. image_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_07.png"
43. apply_single_threshold_with_original(image_path, threshold_value=50,
44. threshold_type=cv2.THRESH_BINARY_INV)
```

## INFERENCE:

**Highlights what's important:** Thresholding picks out bright or dark parts, making them stand out from the rest of the image, which you can see compared to the original.

---



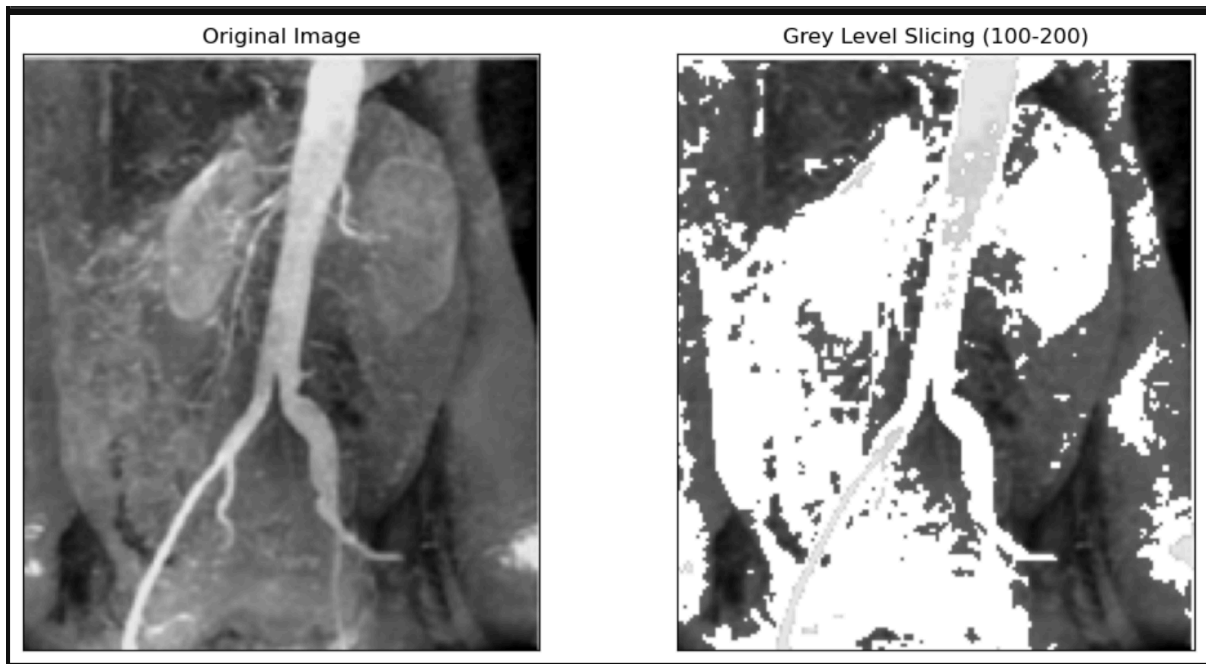


### c. Image Grey Level Slicing with Background - Fig. 8

#### CODE:

```
1. import cv2
2. import matplotlib.pyplot as plt
3. import numpy as np
4.
5. def grey_level_slicing_with_background(image_path, min_range, max_range):
6.     """
7.     Performs grey level slicing on an image, highlighting a specific intensity range while keeping the background.
8.
9.     Args:
10.         image_path (str): Path to the input image.
11.         min_range (int): Minimum intensity value of the slicing range.
12.         max_range (int): Maximum intensity value of the slicing range.
13.     """
14.     try:
15.         img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
16.         if img is None:
17.             raise FileNotFoundError(f"Could not read image at {image_path}")
18.
19.         sliced_image = np.copy(img) # Create a copy to avoid modifying the original
20.         rows, cols = sliced_image.shape
21.
22.         for i in range(rows):
23.             for j in range(cols):
24.                 if min_range <= sliced_image[i, j] <= max_range:
25.                     sliced_image[i, j] = 255 # Highlight the sliced range with white
26.                 else:
27.                     sliced_image[i, j] = img[i, j] #keep the original value.
28.
29.         plt.figure(figsize=(12, 6))
30.
31.         plt.subplot(1, 2, 1)
32.         plt.imshow(img, cmap='gray')
33.         plt.title('Original Image')
34.         plt.xticks([], plt.yticks([]))
35.
36.         plt.subplot(1, 2, 2)
37.         plt.imshow(sliced_image, cmap='gray')
38.         plt.title(f'Grey Level Slicing ({min_range}-{max_range})')
39.         plt.xticks([], plt.yticks([]))
40.
41.         plt.show()
42.
43.     except FileNotFoundError as e:
44.         print(e)
```

```
45. except Exception as e:
46.     print(f"An error occurred: {e}")
47.
48. # Example usage:
49. image_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_08.png"
50. grey_level_slicing_with_background(image_path, 100, 200) # Adjust min and max ranges as needed
```



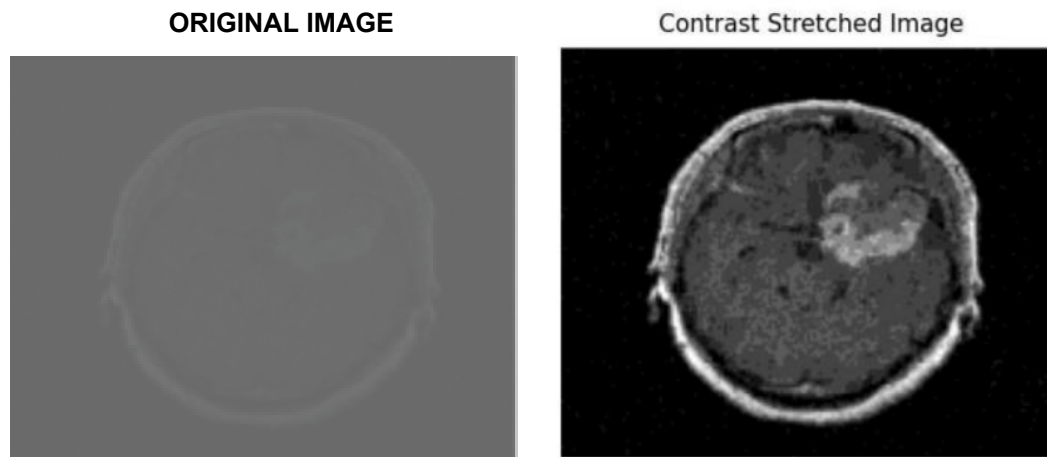
**Focuses on certain brightness levels:** It lets you pick out parts of the image that are within a specific range of brightness (or darkness), making them stand out.

#### d. Contrast Stretching - Fig. 9

##### CODE:

```
1. import cv2
2. import numpy as np
3. from matplotlib import pyplot as plt
4.
5. def contrast_stretch(image_path):
6.     # Load the image in grayscale
7.     image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.
9.     # Check if image loaded successfully
10.    if image is None:
11.        print(f"Error: Could not load image from {image_path}")
12.        print("Please verify:")
13.        print("1. The file exists at this path")
14.        print("2. You have proper read permissions")
15.        print("3. The file is not corrupted")
16.        return
17.
18.    # Find min and max pixel values
19.    min_pixel = np.min(image)
20.    max_pixel = np.max(image)
21.
22.    # Perform contrast stretching
23.    stretched_image = (image - min_pixel) * (255.0 / (max_pixel - min_pixel))
24.    stretched_image = np.uint8(stretched_image)
25.
26.    # Display comparison
27.    plt.figure(figsize=(10, 5))
28.
```

```
29. plt.subplot(1, 2, 1)
30. plt.imshow(image, cmap='gray')
31. plt.title("Original Image")
32. plt.axis("off")
33.
34. plt.subplot(1, 2, 2)
35. plt.imshow(stretched_image, cmap='gray')
36. plt.title(f"Contrast Stretched\nMin: {min_pixel}, Max: {max_pixel}")
37. plt.axis("off")
38.
39. plt.tight_layout()
40. plt.show()
41.
42. # Save the result
43. output_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_09_contrast_stretched.jpg"
44. cv2.imwrite(output_path, stretched_image)
45. print(f"Stretched image saved to: {output_path}")
46.
47. # Run with your specified path
48. contrast_stretch(r"C:\Users\ajay__sr\Downloads\bio med A2\fig_09.png")
```



**Inference:** Contrast stretching expands the image's dynamic range from original min/max values to 0-255, enhancing visibility of subtle details in medical images while preserving relative intensity relationships.

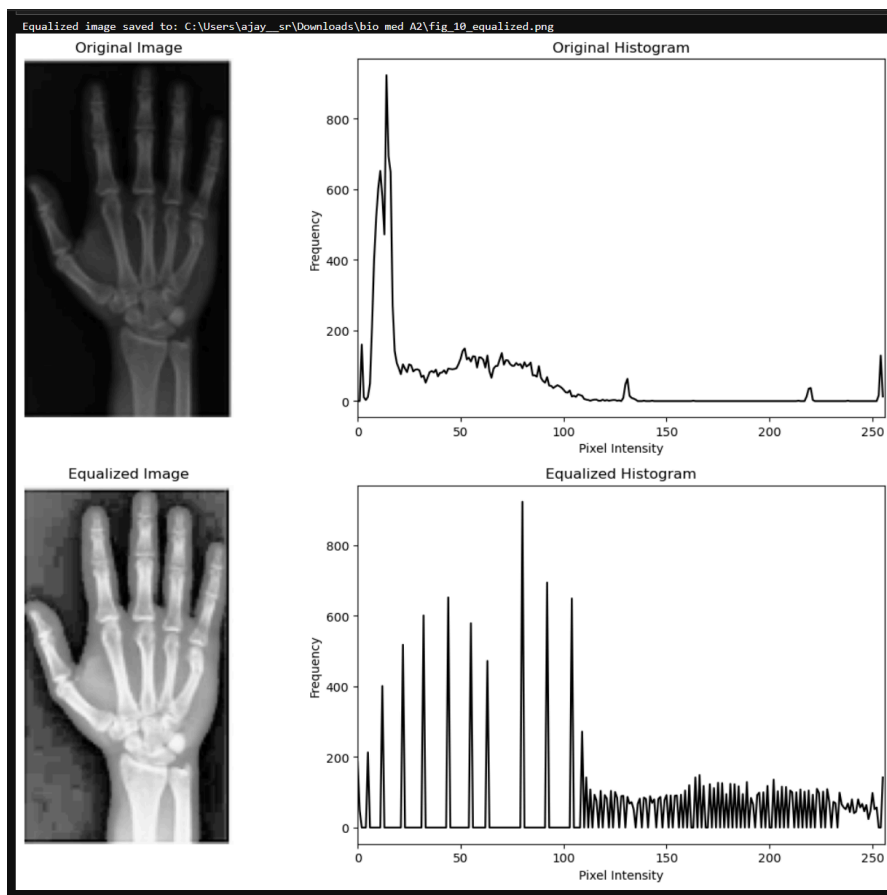
---

## Histogram Processing Fig. 10

### CODE:

```
1. import cv2
2. import numpy as np
3. import matplotlib.pyplot as plt
4.
5. def process_histogram(image_path):
6.     # Load the image in grayscale
7.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if img is None:
9.         print(f"Error: Could not load image from {image_path}")
10.        return
11.
12.    # Calculate histogram of original image
13.    hist_original = cv2.calcHist([img], [0], None, [256], [0, 256])
14.
15.    # Perform histogram equalization
16.    img_equalized = cv2.equalizeHist(img)
17.    hist_equalized = cv2.calcHist([img_equalized], [0], None, [256], [0, 256])
18.
19.    # Create figure with 2x2 subplots
```

```
20. plt.figure(figsize=(12, 10))
21. # Original Image
22. plt.subplot(2, 2, 1)
23. plt.imshow(img, cmap='gray')
24. plt.title('Original Image')
25. plt.axis('off')
26. # Original Histogram
27. plt.subplot(2, 2, 2)
28. plt.plot(hist_original, color='black')
29. plt.title('Original Histogram')
30. plt.xlim([0, 256])
31. plt.xlabel('Pixel Intensity')
32. plt.ylabel('Frequency')
33. # Equalized Image
34. plt.subplot(2, 2, 3)
35. plt.imshow(img_equalized, cmap='gray')
36. plt.title('Equalized Image')
37. plt.axis('off')
38. # Equalized Histogram
39. plt.subplot(2, 2, 4)
40. plt.plot(hist_equalized, color='black')
41. plt.title('Equalized Histogram')
42. plt.xlim([0, 256])
43. plt.xlabel('Pixel Intensity')
44. plt.ylabel('Frequency')
45. plt.tight_layout()
46. # Save the equalized image
47. output_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_10_equalized.png"
48. cv2.imwrite(output_path, img_equalized)
49. print(f"Equalized image saved to: {output_path}")
50. plt.show() # Process the specified image
51. process_histogram(r"C:\Users\ajay_sr\Downloads\bio med A2\fig_10.png")
```



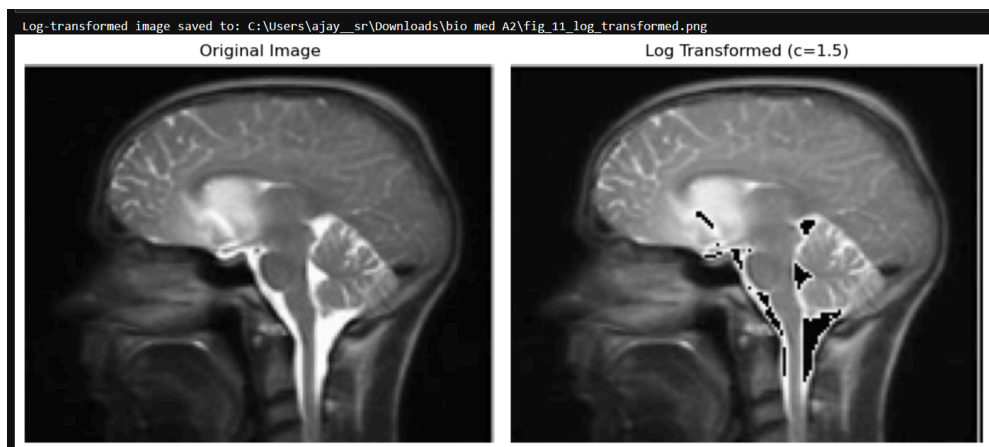
**Histogram equalization** enhances image contrast by redistributing pixel intensities across the full dynamic range (0–255), making subtle anatomical features in medical images more visible.

## Intensity Transformation Operations

### a. Log transformation - Fig. 11

#### CODE:

```
1. import cv2
2. import numpy as np
3. import matplotlib.pyplot as plt
4.
5. def log_transform(image_path):
6.     # Load image
7.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if img is None:
9.         print(f"Error: Could not load image from {image_path}")
10.        return
11.    # Convert to float and normalize
12.    img_float = np.float32(img) / 255.0
13.    # Log transformation (c*log(1 + r))
14.    c = 1.5 # Contrast control parameter
15.    log_transformed = c * np.log1p(img_float) # log1p computes log(1 + x)
16.    log_transformed = np.uint8(log_transformed * 255)
17.    # Display results
18.    plt.figure(figsize=(10, 5))
19.
20.    plt.subplot(1, 2, 1)
21.    plt.imshow(img, cmap='gray')
22.    plt.title('Original Image')
23.    plt.axis('off')
24.
25.    plt.subplot(1, 2, 2)
26.    plt.imshow(log_transformed, cmap='gray')
27.    plt.title(f'Log Transformed (c={c})')
28.    plt.axis('off')
29.
30.    plt.tight_layout()
31.
32.    # Save output
33.    output_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_11_log_transformed.png"
34.    cv2.imwrite(output_path, log_transformed)
35.    print(f"Log-transformed image saved to: {output_path}")
36.
37.    plt.show()
38.
39. # Process the image
40. log_transform(r"C:\Users\ajay_sr\Downloads\bio med A2\fig_11.png")
```

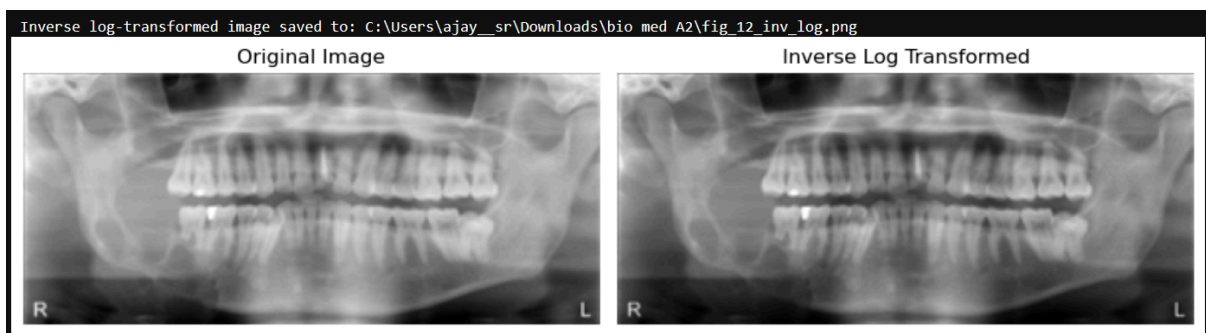


**Inference:** Log transformation enhances subtle details in darker regions of medical images while compressing bright areas, improving visualization of low-contrast anatomical structures.

## b. Inverse log - Fig. 12

### CODE:

```
1. import cv2
2. import numpy as np
3. import matplotlib.pyplot as plt
4.
5. def inverse_log_transform(image_path):
6.     # Load the image
7.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if img is None:
9.         print(f"Error: Could not load image from {image_path}")
10.        return
11.
12.    # Normalize to [0, 1] and add small constant to avoid log(0)
13.    img_normalized = np.float32(img) / 255.0 + 1e-6 # 1e-6 prevents log(0)
14.
15.    # Inverse log transformation (exponential)
16.    c = 1.0 # Scaling constant
17.    inv_log_transformed = c * (np.exp(img_normalized) - 1.0) # exp(r) - 1
18.    inv_log_transformed = np.uint8(255 * inv_log_transformed / np.max(inv_log_transformed))
19.
20.    # Display results
21.    plt.figure(figsize=(10, 5))
22.    plt.subplot(1, 2, 1)
23.    plt.imshow(img, cmap='gray')
24.    plt.title('Original Image')
25.    plt.axis('off')
26.
27.    plt.subplot(1, 2, 2)
28.    plt.imshow(inv_log_transformed, cmap='gray')
29.    plt.title('Inverse Log Transformed')
30.    plt.axis('off')
31.
32.    plt.tight_layout()
33.
34.    # Save output
35.    output_path = r"C:\Users\ajay__sr\Downloads\bio med A2\fig_12_inv_log.png"
36.    cv2.imwrite(output_path, inv_log_transformed)
37.    print(f"Inverse log-transformed image saved to: {output_path}")
38.    plt.show()
39.
40.    # Process the image
41.    inverse_log_transform(r"C:\Users\ajay__sr\Downloads\bio med A2\fig_12.png")
```



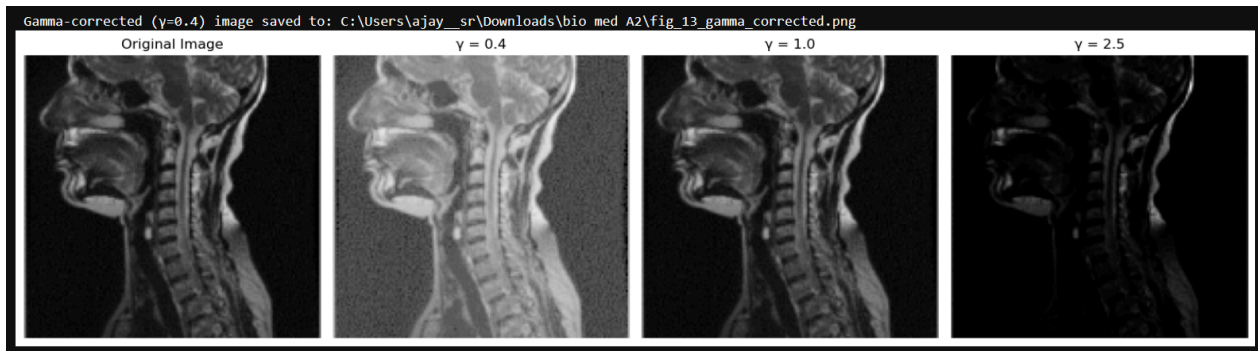
**Inverse log transform** enhances bright-region visibility in medical images while compressing dark areas, complementing standard log transformation.

---

c. Power-law (gamma) transformation - Fig. 13

CODE:

```
1. import cv2
2. import numpy as np
3. import matplotlib.pyplot as plt
4.
5. def gamma_correction(image_path):
6.     # Load the image
7.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
8.     if img is None:
9.         print(f"Error: Could not load image from {image_path}")
10.        return
11.
12.    # Normalize to [0,1] range
13.    img_normalized = np.float32(img) / 255.0
14.
15.    # Define gamma values to test
16.    gamma_values = [0.4, 1.0, 2.5] # <1 (brighten), 1 (no change), >1 (darken)
17.
18.    # Create figure
19.    plt.figure(figsize=(15, 5))
20.
21.    # Original image
22.    plt.subplot(1, len(gamma_values)+1, 1)
23.    plt.imshow(img, cmap='gray')
24.    plt.title('Original Image')
25.    plt.axis('off')
26.
27.    # Apply and display gamma corrections
28.    for i, gamma in enumerate(gamma_values, 2):
29.        # Power-law transformation
30.        gamma_corrected = np.power(img_normalized, gamma)
31.        gamma_corrected = np.uint8(gamma_corrected * 255)
32.
33.        # Save the most useful transformation (gamma=0.4 for dark images)
34.        if gamma == 0.4:
35.            output_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_13_gamma_corrected.png"
36.            cv2.imwrite(output_path, gamma_corrected)
37.            print(f"Gamma-corrected (gamma={gamma}) image saved to: {output_path}")
38.
39.        # Display
40.        plt.subplot(1, len(gamma_values)+1, i)
41.        plt.imshow(gamma_corrected, cmap='gray')
42.        plt.title(f'gamma = {gamma}')
43.        plt.axis('off')
44.
45.    plt.tight_layout()
46.    plt.show()
47.
48.    # Process the image
49.    gamma_correction(r"C:\Users\ajay_sr\Downloads\bio med A2\fig_13.png")
```



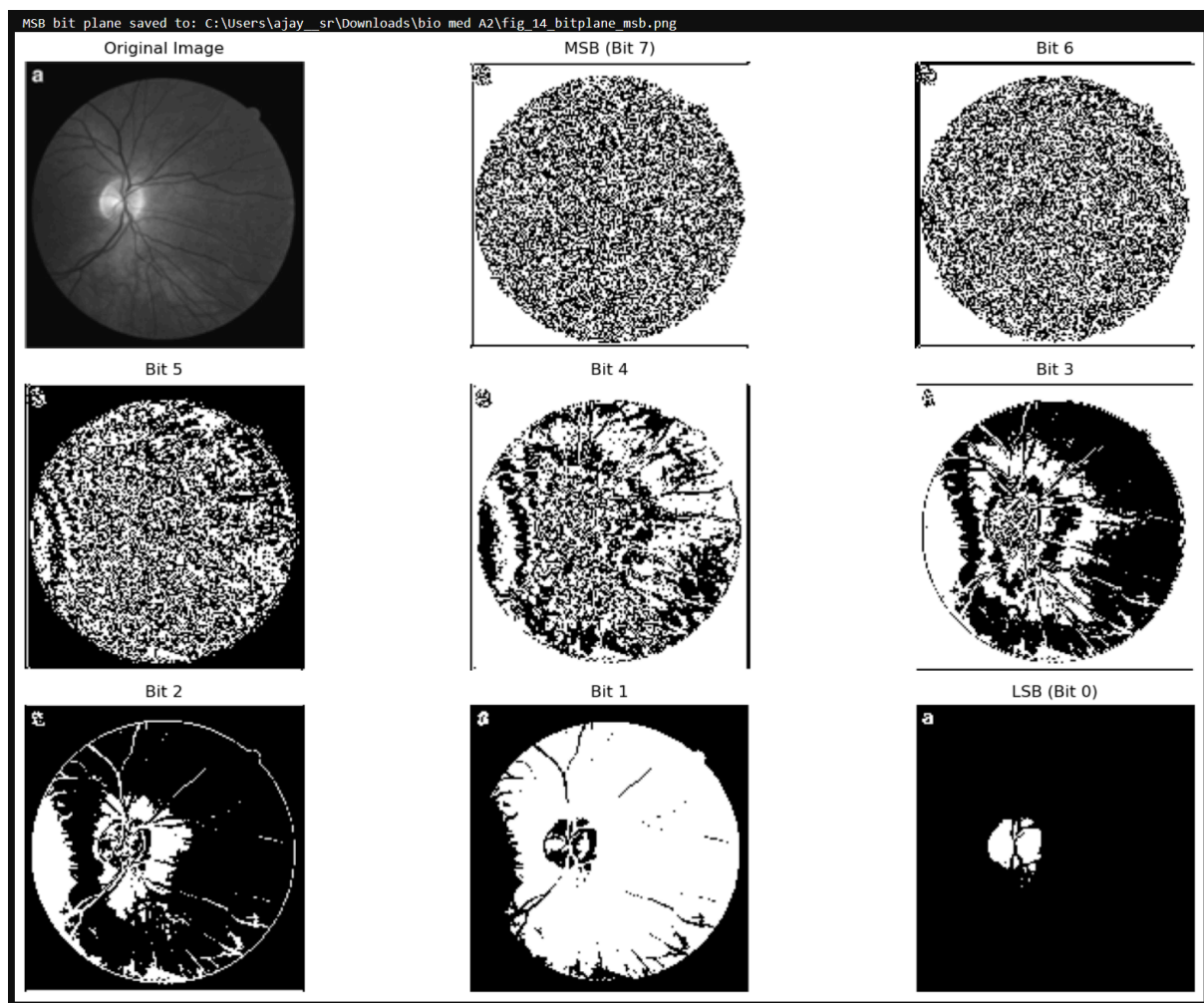
**Gamma correction** ( $\gamma < 1$ ) optimally enhances dark-region visibility in medical images while preserving bright-area details, adjustable based on diagnostic needs.

## Bit-Plane Slicing – Fig. 14

### CODE:

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3.
4. def bit_plane_slicing(image_path):
5.     # Load image
6.     img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
7.     if img is None:
8.         print(f"Error: Could not load image from {image_path}")
9.         return
10.
11.     # Get image dimensions
12.     height, width = img.shape
13.
14.     # Create 8-bit planes
15.     bit_planes = []
16.     for i in range(8):
17.         plane = np.zeros((height, width), dtype=np.uint8)
18.         # Extract the i-th bit plane
19.         plane[:, :] = (img >> i) & 1
20.         plane *= 255 # Scale to 0-255 for visualization
21.         bit_planes.append(plane)
22.
23.     # Display original and bit planes
24.     plt.figure(figsize=(15, 10))
25.
26.     # Original image
27.     plt.subplot(3, 3, 1)
28.     plt.imshow(img, cmap='gray')
29.     plt.title('Original Image')
30.     plt.axis('off')
31.     # Display each bit plane (MSB to LSB)
32.     titles = ['MSB (Bit 7)', 'Bit 6', 'Bit 5',
33.              'Bit 4', 'Bit 3', 'Bit 2',
34.              'Bit 1', 'LSB (Bit 0)']
35.     for i in range(8):
36.         plt.subplot(3, 3, i+2)
37.         plt.imshow(bit_planes[i], cmap='gray')
38.         plt.title(titles[i])
39.         plt.axis('off')
40.     plt.tight_layout()
41.     # Save the most significant bit planes (MSB)
42.     output_path = r"C:\Users\ajay_sr\Downloads\bio med A2\fig_14_bitplane_msb.png"
43.     cv2.imwrite(output_path, bit_planes[7]) # MSB (Bit 7)
44.     print(f"MSB bit plane saved to: {output_path}")
45.     plt.show()
46.     # Process the image
47.     bit_plane_slicing(r"C:\Users\ajay_sr\Downloads\bio med A2\fig_14.png")
```





#### MSB (Bit 7):

- Carries 90% of visual information (structural details).
- Looks like a recognizable but coarser version of the original.
- *Use:* For image compression (preserving only MSB saves space).

#### Mid-Range Bits (6-4):

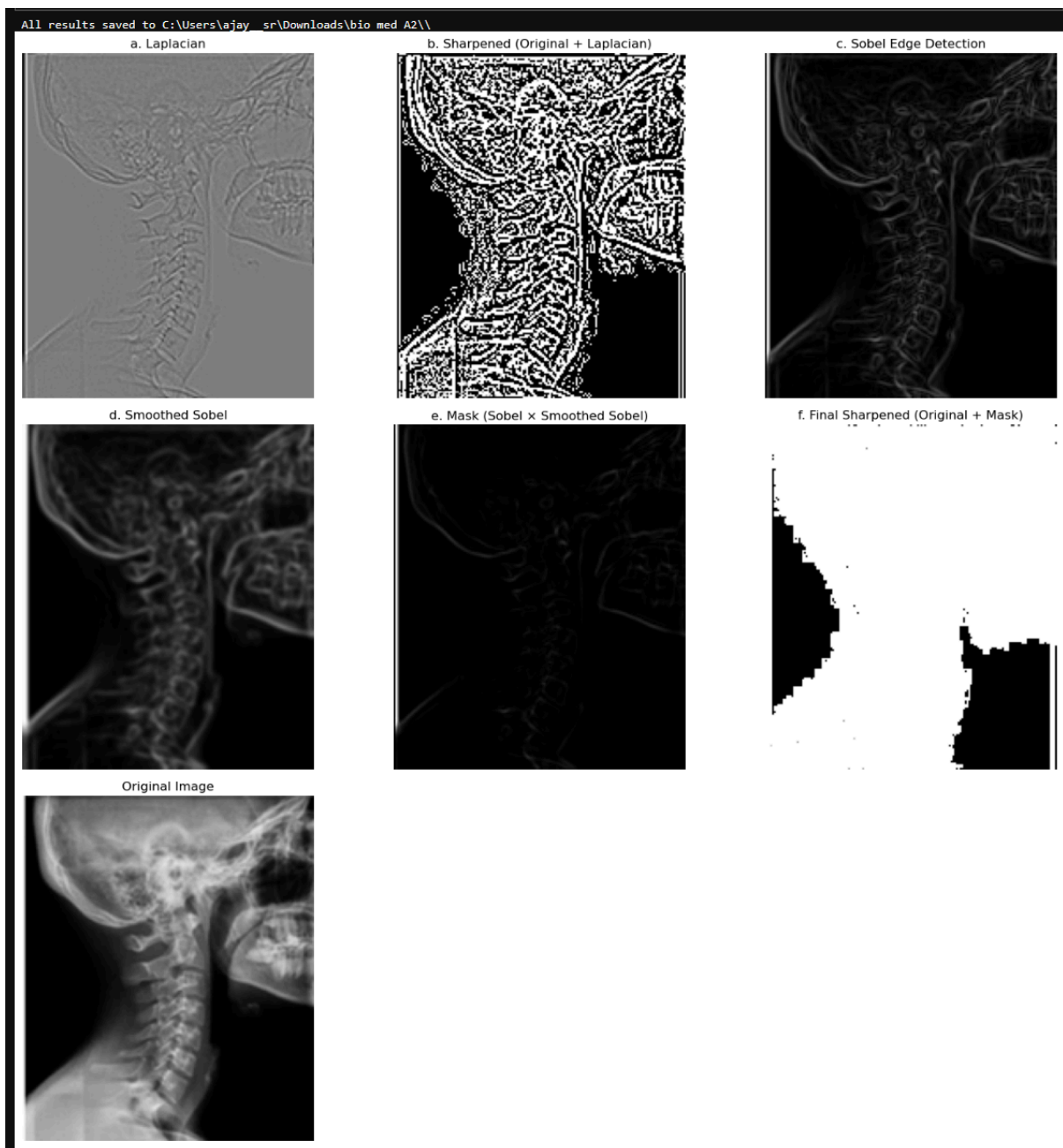
- Contain textural details (e.g., tissue patterns).
- Gradually lose structural clarity from Bit 6 → Bit 4.

#### LSB (Bit 0-3):

- Mostly noise and subtle gradients.
  - Appears like random speckles (least diagnostic value).
  - *Use:* Noise analysis or steganography.
-

## Image Masking -Fig.15

- Laplacian of the image Fig. 15
- Sharpen image adding original image and output of a
- Sobel of original image
- Smoothing of output of c
- Mask image formed by product of c and d
- Sharpen the image by sum of original and f



## A. Laplacian Filter

**What Happens:** Detects rapid intensity changes (edges).

**Inference:** "Reveals all edges but also amplifies noise."

## B. Sharpened (Original + Laplacian)

**What Happens:** Enhances edges by subtracting Laplacian from the original.

**Inference:** "Boosts edges but may over-sharpen small details."

## C. Sobel Edge Detection

**What Happens:** Highlights horizontal/vertical edges.

**Inference:** "Shows clear structural outlines with less noise."

## D. Smoothed Sobel

**What Happens:** Blurs the Sobel result to reduce noise.

**Inference:** "Retains major edges while smoothing out noise."

## E. Mask (Sobel × Smoothed Sobel)

**What Happens:** Combines edge and smoothed edge images.

**Inference:** "Creates a focused mask of important edges."

## F. Final Sharpened (Original + Mask)

**What Happens:** Adds the mask back to the original image..

**Inference:** "Best result: clearer anatomy with natural look."

---